SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Design Task

Code of Conduct:

I M. MUKKARAM(192524357) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
M. MUKKARAM

Annexure A

Design Task

1. **Hybrid AI Recommendation Engine Design**

   Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. **Smart Disaster Detection System**

   Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. **AI-Based Fraud Detection Framework**

   Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. **Autonomous Supply Chain Optimization System**

   Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. **Personalized Learning Analytics Platform**

   Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

QUESTION 1: HYBRID AI RECOMMENDATION ENGINE DESIGN

ANSWER:

A Hybrid AI Recommendation Engine is a recommendation system that combines content-based filtering and collaborative filtering to provide accurate and personalized recommendations. The system can use PySpark RDDs to process large amounts of user-item interaction data in a distributed environment.

The content-based filtering method recommends items based on the characteristics of items that a user has previously liked. For example, if a user frequently watches science-fiction movies, the system can recommend other science-fiction movies. Collaborative filtering recommends items by identifying similarities between users and their preferences. If two users have similar interests, items liked by one user can be recommended to the other.

PySpark RDDs are useful because user-item interaction data can be very large. The data can be divided into partitions and processed simultaneously across multiple nodes. RDD transformations such as map(), filter(), flatMap(), reduceByKey(), and groupByKey() can be used to process the interaction data. Actions such as collect(), count(), and reduce() can be used to obtain results.

The basic processing flow is:

User-Item Data → PySpark RDD → Data Cleaning → Content-Based Filtering + Collaborative Filtering → Hybrid Recommendation → Intelligent Agent → User Feedback → Updated Recommendation

Intelligent agents play an important role in adapting recommendations. The agent observes user behaviour such as clicks, searches, ratings, purchases, and viewing history. It analyzes this feedback and modifies future recommendations according to the user's changing interests.

The hybrid approach provides better recommendations because it combines item characteristics with information from similar users. Distributed processing using PySpark makes the system suitable for large-scale applications. Intelligent agents make the recommendation process adaptive and capable of responding to user behaviour in real time.

CONCLUSION:

The Hybrid AI Recommendation Engine provides scalable and personalized recommendations by combining PySpark RDD-based distributed processing, content-based filtering, collaborative filtering, and intelligent agents. It can efficiently process large-scale user-item data and continuously improve recommendations using user feedback.

QUESTION 2: SMART DISASTER DETECTION SYSTEM

ANSWER:

A Smart Disaster Detection System is a distributed AI system designed to detect natural disasters such as floods and earthquakes at an early stage. The system collects information from multiple sources, including satellite images, environmental sensors, and social media.

The system architecture consists of several major components:

Data Sources → Data Collection → PySpark RDD Processing → Data Fusion → AI Detection → Intelligent Agents → Early Warning System

Satellite images provide information about geographical changes, water levels, damaged areas, and other environmental conditions. Sensors provide real-time information such as rainfall, water level, temperature, vibration, and pressure. Social media can provide reports and information from people in affected areas.

PySpark can process these large and continuously generated datasets using RDDs. The incoming data is divided into partitions and processed across multiple nodes. RDD transformations such as map(), filter(), flatMap(), and reduceByKey() can be used for data cleaning, feature extraction, and aggregation.

Data fusion combines information from different sources to improve disaster detection. For example, if sensors indicate increasing water levels, satellite images show expanding water coverage, and social media reports mention flooding, the system can combine these signals to identify a possible flood.

Intelligent agents coordinate different activities in the system. A monitoring agent continuously observes incoming data. A detection agent analyzes the data and identifies possible disasters. A warning agent generates alerts when the disaster probability exceeds a specified level. A response agent can recommend suitable response strategies.

The system can provide early warnings through communication channels such as mobile applications, emergency messages, or control centers. Distributed processing allows the system to handle large amounts of data quickly.

CONCLUSION:

The Smart Disaster Detection System combines PySpark, RDD operations, multi-source data fusion, AI techniques, and intelligent agents. The system can improve early detection, provide faster warnings, and support effective disaster response.

QUESTION 3: AI-BASED FRAUD DETECTION FRAMEWORK

ANSWER:

An AI-Based Fraud Detection Framework is a scalable system that identifies suspicious financial transactions using artificial intelligence and distributed computing. Since financial institutions process millions of transactions, a distributed system is required to analyze the data efficiently.

The basic architecture of the system is:

Transaction Data → PySpark RDD → Data Preprocessing → Feature Extraction → Distributed Anomaly Detection → Intelligent Agents → Fraud Alert → Continuous Learning

PySpark RDDs can divide large transaction datasets into smaller partitions and process them in parallel. RDD transformations such as map(), filter(), and reduceByKey() can be used to clean data, calculate transaction statistics, and identify suspicious patterns.

Features such as transaction amount, transaction frequency, location, transaction time, account history, and unusual spending behaviour can be used for fraud detection.

Anomaly detection algorithms identify transactions that are significantly different from normal behaviour. For example, if a customer normally makes small transactions from one location but suddenly makes several large transactions from different locations, the system can identify this behaviour as suspicious.

Intelligent agents can collaborate to improve fraud detection. A monitoring agent observes transactions continuously. An analysis agent evaluates suspicious transactions. A decision agent determines whether an alert should be generated. A learning agent uses feedback from previously confirmed fraudulent and genuine transactions to improve future detection.

The system can generate alerts for suspicious transactions and send them to appropriate security or banking personnel. Distributed processing helps the system handle a large number of transactions with reduced processing time.

The system should also consider false positives, data privacy, security, and model accuracy. Continuous learning can help the system adapt to new fraud patterns.

CONCLUSION:

The AI-Based Fraud Detection Framework combines PySpark RDDs, distributed anomaly detection, and intelligent agents to identify fraudulent activities efficiently. It is scalable and can continuously improve its detection capability using feedback and newly available transaction data.

QUESTION 4: AUTONOMOUS SUPPLY CHAIN OPTIMIZATION SYSTEM

ANSWER:

An Autonomous Supply Chain Optimization System is a distributed AI system that uses artificial intelligence and PySpark to improve inventory management, logistics, and demand forecasting. The system helps organizations reduce costs and improve operational efficiency.

The system architecture can be represented as:

Supply Chain Data → PySpark RDD → Data Processing → Demand Forecasting → Inventory and Logistics Optimization → Intelligent Agents → Decentralized Decisions

The system collects data related to inventory levels, sales, suppliers, transportation, delivery times, customer demand, and product availability. PySpark RDDs can process this large-scale data in parallel.

RDD transformations such as map(), filter(), groupByKey(), and reduceByKey() can be used to transform and aggregate supply chain information. For example, sales records can be grouped according to products and locations to determine demand patterns.

Demand forecasting techniques can predict future requirements based on historical sales data. Inventory optimization can determine when and how much stock should be ordered. Logistics optimization can help identify efficient transportation and delivery decisions.

Multiple intelligent agents can work together. An inventory agent monitors stock levels and recommends replenishment. A demand forecasting agent predicts future demand. A logistics agent manages transportation and delivery decisions. A supplier agent monitors supplier availability and delivery performance.

These agents communicate with each other and make decentralized decisions. For example, when an inventory agent detects low stock, it can communicate with the supplier agent and logistics agent to arrange replenishment and transportation.

PySpark provides scalability by distributing data processing across multiple machines. This allows the system to handle large amounts of supply chain data efficiently.

CONCLUSION:

The Autonomous Supply Chain Optimization System uses PySpark RDDs and intelligent agents to process large-scale supply chain data and make decentralized decisions. It can reduce inventory costs, improve logistics, support demand forecasting, and increase overall supply chain efficiency.

QUESTION 5: PERSONALIZED LEARNING ANALYTICS PLATFORM

ANSWER:

A Personalized Learning Analytics Platform is an AI-based educational system that analyzes student learning behaviour and performance data to provide personalized learning experiences. PySpark and RDDs can be used to process large amounts of student data efficiently.

The basic system architecture is:

Student Data → PySpark RDD → Data Preprocessing → Learning Behaviour Analysis → Student Performance Prediction → Intelligent Agent → Personalized Content → Student Feedback

The system can collect data such as quiz scores, assignment marks, attendance, learning time, completed lessons, interaction history, and assessment results. PySpark RDDs can process this data across multiple computing nodes.

RDD transformations such as map(), filter(), groupByKey(), and reduceByKey() can be used to clean, transform, and aggregate student data. For example, student performance can be grouped according to subjects or learning activities.

The platform analyzes learning behaviour to identify the strengths and weaknesses of individual students. If a student performs poorly in a particular topic, the system can recommend additional learning materials, practice questions, videos, or simpler explanations.

Intelligent agents support adaptive learning. A learning agent observes student behaviour and performance. A recommendation agent selects suitable learning content. A feedback agent provides real-time suggestions based on student responses. A monitoring agent tracks learning progress.

The platform can support thousands of learners because PySpark distributes data processing across multiple machines. This makes the system scalable and suitable for large educational institutions.

The system can continuously improve personalization by using feedback from students. For example, if a student completes recommended content successfully, the system can update the student's learning profile and recommend more advanced content.

CONCLUSION:

The Personalized Learning Analytics Platform combines PySpark, RDD operations, AI-based analytics, and intelligent agents to provide adaptive education. It can analyze large-scale student data, identify individual learning needs, provide real-time feedback, and personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration

Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation